

Integração com API

API (Application Programming Interface) é um conjunto de regras e protocolos que permite a interação e comunicação entre diferentes softwares. Em termos simples, uma API define como os componentes de software devem interagir entre si.

Uma API atua como uma ponte entre aplicativos, permitindo que eles compartilhem dados, funcionalidades e serviços de maneira padronizada. Ela define um conjunto de métodos, protocolos e formatos de dados que os desenvolvedores podem utilizar para solicitar e receber informações ou realizar ações específicas em um sistema ou serviço.

As APIs podem ser usadas para uma ampla variedade de propósitos, como integração de sistemas, desenvolvimento de aplicativos móveis, acesso a serviços web, automação de tarefas e muito mais. Elas fornecem uma maneira eficiente de reutilizar e compartilhar funcionalidades entre diferentes aplicativos e plataformas.

No React, uma API pode ser consumida usando chamadas assíncronas para buscar dados e atualizar o estado de um componente. Existem várias maneiras de consumir uma API, vamos começar vendo sobre o [fetch](#).

Fetch

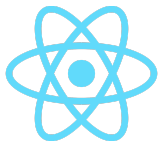
No React, o [fetch](#) é uma função global do JavaScript que permite fazer requisições HTTP assíncronas. Ele é usado para enviar solicitações para um servidor e receber respostas contendo dados.

O retorno dessa função é uma [Promise](#), que resolve para um objeto Response, que contém informações sobre a resposta, como status, cabeçalhos e corpo.

Veja um exemplo:

```
fetch('https://api.com/endpoint') // url da api
  .then(response => response.json()) // extrair os dados
  .then(data => {
    // manipular os dados retornados
  })
  .catch(error => {
    // lidar com erros
  });
```

No exemplo acima estamos fazendo uma requisição para a URL <https://api.com/endpoint>. Como o `fetch()` retorna uma promise, estamos tratando com o `then()` e o `catch()`.



O `fetch` é bastante flexível e pode ser usado para diferentes tipos de solicitações, como GET, POST, PUT e DELETE, passando as opções adequadas. Por padrão, o `fetch` realiza solicitações GET, mas você pode especificar outros métodos através das opções. Veja abaixo um exemplo de uma requisição utilizando o POST como método:

```
fetch('https://api.com/endpoint', { // url da api
  method: 'POST', // define o método HTTP
  headers: {
    'Content-Type': 'application/json', // indica que o corpo da
    solicitação é um um objeto json
  },
  body: JSON.stringify({ nome: 'Pedro', idade: 25 }), // objeto sendo
  convertido em string
})
  .then(response => response.json()) // extrair os dados
  .then(data => {
    // manipular os dados retornados
  })
  .catch(error => {
    // tratar os erros
  });
```

Lembre-se de que o `fetch` é uma função [assíncrona](#), o que significa que você pode usar as palavras-chave `async` e `await` para escrever um código mais conciso e legível ao fazer chamadas assíncronas no React.

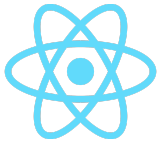
Axios

O [Axios](#) é uma biblioteca amplamente utilizada para realizar solicitações HTTP em JavaScript, inclusive no React. Essa biblioteca oferece uma interface simples e fácil de usar para realizar chamadas assíncronas a APIs, além de oferecer suporte a recursos avançados, como interceptação de requisições, tratamento de erros e conversão automática dos dados de resposta.

Para fazer o uso do axios, é preciso instalar a biblioteca no seu projeto:

```
npm install axios
```

Caso você esteja usando o yarn como o gerenciador de pacotes, rode esse comando:



```
yarn add axios
```

Após isso, faça a importação do axios no seu componente:

```
import axios from 'axios';
```

O seu uso é muito semelhante ao fetch. Veja o exemplo abaixo de uma requisição GET utilizando a função `axios.get()`:

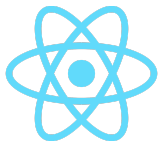
```
axios.get('https://api.com/endpoint')
  .then(response => {
    // manipular os dados retornados
  })
  .catch(error => {
    // tratar os erros
  });
```

Você também pode fazer solicitações POST, PUT, DELETE, entre outras, com o Axios. Aqui está um exemplo de solicitação POST usando o Axios:

```
axios.post('https://api.com/endpoint', { nome: Pedro, idade: 25 })
  .then(response => {
    // manipular os dados retornados

  })
  .catch(error => {
    console.log(error);
  });
```

O Axios simplifica a interação com APIs no React, fornecendo uma sintaxe limpa e várias funcionalidades úteis.



useEffect e requisições

O uso do [useEffect](#) para realizar as chamadas de requisição é uma prática comum no React, especialmente quando você deseja buscar dados quando o componente é montado, atualizado ou desmontado. O useEffect é um hook que permite que você execute efeitos colaterais em componentes funcionais, como buscar dados de uma API.

Veja um exemplo de uma requisição sendo chamada dentro do hook useEffect:

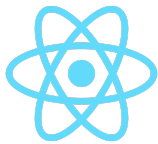
```
import { useEffect, useState } from 'react';
import axios from 'axios';

function Componente() {
  const [dados, setDados] = useState([]); // estado para armazenar dados da API

  useEffect(() => {
    axios.get('https://api.com/endpoint')
      .then(response => {
        setDados(response.data); // inserindo os dados retornados da API em nosso estado
      })
      .catch(error => {
        throw error; // Lançando o erro para frente
      });
  }, []);

  return (
    <div>
      // mostrando os dados do estado
      {dados.map(item => (
        <p key={item.id}>{item.nome}</p>
      ))}
    </div>
  );
}
```

Ao realizar a chamada de requisição dentro do `useEffect` com um array de dependências vazio (`[]`), garantimos que a requisição será feita somente uma vez, quando o componente for “montado”.



Porém, você pode fazer chamadas de requisição em outros lugares do seu componente, se necessário. Por exemplo, você pode fazer uma chamada de requisição em resposta a um evento, como um clique em um botão, ou em uma função específica acionada por alguma lógica de negócio. Veja o exemplo:

```
import { useState } from 'react';
import axios from 'axios';

function Componente() {
  const [dados, setDados] = useState([]);

  const buscarDados = () => {
    axios.get('https://api.com/endpoint')
      .then(response => {
        setDados(response.data);
      })
      .catch(error => {
        console.log(error);
      });
  };

  return (
    <div>
      <button onClick={buscarDados}>Buscar Dados</button>
      {dados.map(item => (
        <p key={item.id}>{item.nome}</p>
      ))}
    </div>
  );
}
```

Nesse caso, a função `buscarDados`, apenas será chamada quando o botão 'Buscar Dados' for clicado.

Axios ou Fetch?

Resumidamente, o Axios é uma biblioteca de terceiros que oferece uma interface mais amigável e recursos adicionais, como transformação automática de dados e interceptação de requisições. O Fetch, por outro lado, é uma API nativa do navegador que é mais leve e amplamente suportada pelos navegadores modernos.

A escolha entre Axios e Fetch depende das necessidades específicas do seu projeto e das preferências da equipe de desenvolvimento! O Axios é frequentemente usado em projetos React devido à sua popularidade e integração fácil com o ecossistema React.